

HW2 EDA Script Walkthrough

Part 2B — Explain the Code and Output (new Claude Cowork session)

Author: Hannah Dickinson

Course: MIS3060 Business Intelligence with AI — Villanova University

Script reviewed: hw02_eda.py

Prompt 1 — Code Walkthrough

Question asked: “Walk me through each section of this script, including the grouping, correlation, and charting steps. What should I see in the terminal when I run it? List each expected output value explicitly.”

Note: this session was not given the data file, so it could not run the script. Anywhere a value depends on the actual data, the response used a bracketed placeholder instead of a real number.

Setup (lines before section 1)

The `pd.set_option` calls turn off pandas' truncation so `describe()`, `value_counts()`, etc. print in full rather than being cut off with The `log()` helper does double duty: it prints to the terminal and appends the same string to `profile_lines`, which becomes `hw02_profile.txt` at the end. `section()` prints a blank line, a row of 70 = characters, the title, and another row of =, so every section is visually boxed in the terminal.

1. Load the data

Reads the CSV into `df`. Nothing prints here — if the path is wrong you'd see a `FileNotFoundError` instead of section 2.

2. Dataset shape

The spec's expected shape is (298772, 9) (see section 14), so if the file matches spec you'd see Rows : 298,772 | Columns: 9.

```
=====
2. DATASET SHAPE
=====
Rows: [N,NNN] | Columns: [N]
```

3. Column names and data types

Prints `df.dtypes` as plain text — one line per column, column name padded, then its dtype (int64, float64, object, etc.). The exact column names/order/dtypes depend on the real CSV — this is illustrative based on what the rest of the script references.

```
=====
3. COLUMN NAMES AND DATA TYPES
=====
txn_id          int64
client_id       int64
advisor_id      int64
```

```

security_id    int64
txn_date       object
txn_type       object
shares         float64
price          float64
amount         float64

```

4. Missing values per column

`df.isnull().sum()` — one line per column with its null count.

```

=====
4. MISSING VALUES PER COLUMN
=====
txn_id         0
client_id      0
...
amount        [N]

```

5. Descriptive statistics

`df.describe()` on numeric columns only — count, mean, std, min, 25%, 50%, 75%, max, one column per numeric field, printed as a table.

6. txn_type value counts and percentages

Builds a two-column table (count, percent) from `value_counts()`, sorted most-to-least frequent by construction (pandas' default), percent rounded to 2 decimals.

```

=====
6. TXN_TYPE VALUE COUNTS AND PERCENTAGES
=====
      count  percent
BUY      [n1]    [p1]
SELL     [n2]    [p2]
DIVIDEND [n3]    [p3]
...

```

7. Unique entity counts

```

=====
7. UNIQUE ENTITY COUNTS
=====
Unique clients:    [N,NNN]
Unique advisors:   [N,NNN]
Unique securities: [N,NNN]

```

8. Transaction date range

Converts `txn_date` to datetime and prints the min/max as ISO dates. Given the header comment says Jan 2020–Dec 2024, you'd expect something close to 2020-01-0X and 2024-12-3X if the data matches the stated range.

```

=====
8. TRANSACTION DATE RANGE
=====
Earliest txn_date: [YYYY-MM-DD]
Latest txn_date:   [YYYY-MM-DD]

```

9. Duplicate txn_id check

`.duplicated().sum()` counts rows whose `txn_id` repeats an earlier one. For a clean fact table keyed on `txn_id`, you'd hope for 0.

```
=====
9. DUPLICATE TXN_ID CHECK
=====
Duplicate txn_id count: [N]
```

10. Amount: mean, median, skewness

Skewness > 0 means a right tail (a few very large transactions pulling the mean above the median), which is typical for transaction/dollar-amount data.

```
=====
10. AMOUNT: MEAN, MEDIAN, SKEWNESS
=====
Mean amount:      [X,XXX.XX]
Median amount:    [X,XXX.XX]
Skewness amount:  [X.XX]
```

11. Grouping — amount by txn_type

This is the first grouping step. `df.groupby("txn_type")["amount"].agg(count=..., mean=..., median=...)` collapses all rows into one row per transaction type, computing three aggregates at once. `mean/median` are then rounded to 2 decimals, and the whole table is re-sorted by mean descending (`sort_values("mean", ascending=False)`) so the highest-average-dollar transaction type appears first. This sorted index is reused later in section 15b to order the box plot categories.

```
=====
11. AMOUNT BY TXN_TYPE (GROUPED)
=====
      count      mean      median
[TYPE_A]  [n]      [high]      [med]
[TYPE_B]  [n]      [...]      [...]
...
```

12. Correlation step

`df[["shares", "price", "amount"]].corr()` computes the pairwise Pearson correlation matrix among those three numeric columns (values between -1 and 1, diagonal always 1.00), rounded to 2 decimals. Then `itertools.combinations` generates the three unique pairs, sorted by absolute correlation value descending, and the top 3 (which, with only 3 columns, is all of them) print as a ranked list. Since amount is typically `shares × price` (or close to it), expect `shares–amount` to be one of the strongest relationships.

```
=====
12. CORRELATION MATRIX (shares, price, amount)
=====
      shares  price  amount
shares    1.00  [r12]  [r13]
price    [r12]    1.00  [r23]
amount    [r13]  [r23]    1.00

Three strongest correlations (excluding self-correlation):
1. shares - amount: [r]
2. [pair]: [r]
3. [pair]: [r]
```

13. Shares min/max/negative count by txn_type

Another groupby, this time with a lambda aggregate ((s < 0).sum()) alongside min/max. This is a data-quality check — negative shares would typically be expected for SELL-type transactions (if shares are signed) but might be a red flag for BUY.

```
=====
13. SHARES: MIN, MAX, NEGATIVE COUNT BY TXN_TYPE
=====
      min_shares  max_shares  negative_count
[TYPE_A]      [...]      [...]      [...]
[TYPE_B]      [...]      [...]      [...]
```

14. Shape validation

Not wrapped in log(), so it prints directly with print() and is not saved to hw02_profile.txt. It only appears at all if df.shape != (298772, 9). If the shape matches, this section produces no output whatsoever — nothing prints, and the script moves straight to the charts.

```
*****
WARNING: Dataset shape (R, C) does not match the expected shape (298772, 9).
Investigate before proceeding with analysis.
*****
```

15. Charting steps

Three PNGs get written to charts/ (created if it doesn't exist), none of which print to the terminal themselves.

15a: a 50-bin histogram of amount with a red dashed line at the mean and green dashed line at the median (values reused from section 10), saved as hist_amount.png.

15b: a horizontal box plot of amount, one box per txn_type, ordered by the section-11 sort (highest mean first), saved as box_amount_by_type.png. It tries the modern tick_labels argument first and falls back to the older labels argument for older matplotlib versions.

15c: a scatter plot of shares vs. amount, one color per txn_type (looped via df["txn_type"].unique(), so legend order follows first-appearance order in the data, not the sorted order from section 11), saved as scatter_shares_amount.png.

After all three are saved, this prints:

```
Charts saved to 'charts/':
- hist_amount.png
- box_amount_by_type.png
- scatter_shares_amount.png
```

16. Save the profile

Writes every line captured by log() — i.e., everything from sections 2 through 13 only, since section 14's warning and section 15's chart-saved messages use bare print() — to hw02_profile.txt, one per line.

```
Profile summary saved to 'hw02_profile.txt'
```

End-to-end summary: the terminal shows sections 2 through 13 in order (boxed headers, one block of output each), then either nothing or a warning block for section 14, then the “Charts saved to...” block, then the final “Profile summary saved to...” line — with hw02_profile.txt on disk containing a plain-text copy of just sections 2–13, and three PNGs sitting in a new charts/ folder.

Prompt 2 — Output Interpretation

Question asked: “Here is the terminal output from running an EDA script on a wealth management transaction dataset. What does each value mean? Flag anything that looks unexpected or that I should investigate before using this data in an analysis.”

This time the complete real terminal output (from actually running `hw02_eda.py`) was pasted in, so the response below discusses real values, not placeholders.

Shape, types, and missingness (sections 2–4)

298,772 rows, 9 columns — matches the spec exactly. `security_id`, `shares`, and `price` are each missing exactly 101,597 values, and that number isn't a coincidence: it's precisely Deposit + Advisory Fee + Withdrawal ($35,981 + 35,766 + 29,850 = 101,597$). So the “missing” values aren't a data quality problem — those three transaction types simply don't involve a security (you don't buy shares when making a deposit), so `security_id/shares/price` are structurally null for them. That's why `security_id` shows as float64 rather than int64, too — pandas can't hold NaN in an integer column, so it silently upcasts to float. If you later use `security_id` as a join key, cast it to a nullable integer type (Int64) first so you don't end up joining on 249.0 vs 249.

One thing worth confirming with whoever built this table: is that null pattern guaranteed by design, or could a real security-linked transaction ever legitimately have a null `security_id`? If it's guaranteed, this isn't a flag at all — just document the assumption.

Descriptive stats (section 5)

`shares` ranges from about -499.63 to 499.997, `price` from exactly 10.00 to exactly 500.00. Those round bounds (10–500 for both price and share count) are a strong signal this is simulated/synthetic data rather than pulled from a live market feed — worth keeping in mind so you don't read too much into, say, the specific skewness value as if it reflected real market behavior.

txn_type breakdown (section 6)

Six categories, percentages sum to 100%, counts sum to the full 298,772 rows — internally consistent, no stray categories.

Entity counts and dates (sections 7–9)

2,700 clients, 25 advisors, 500 securities, date range 2020-01-01 to 2024-12-30 (matches the “Jan 2020–Dec 2024” description in the docstring, just one day short of year-end). Zero duplicate `txn_id` values, so the primary key is clean.

Amount stats (sections 10–11)

Mean (\$54,075) sitting well above the median (\$41,220) with skew of 1.15 is a normal right-skewed shape for dollar amounts — a handful of large transactions pull the mean up. Grouped by type, Dividend/Buy/Sell all average ~\$63–64k with similar medians, and Deposit/Withdrawal both average ~\$50k. Advisory Fee stands out: mean \$7,375 but median only \$859 — the mean is over 8x the median, a much bigger mean/median gap than any other category. That's worth investigating directly — it suggests either a few very large outlier fees inflating the average, or two distinct fee structures (e.g., flat fees vs. AUM-based fees) getting lumped into one category. I'd pull the distribution of Advisory Fee amounts specifically before using

that group's mean in anything.

Correlations (section 12)

shares and price are essentially uncorrelated (0.00), which makes sense if they were generated independently. Both correlate moderately with amount (0.65 and 0.64) — consistent with amount being roughly $\text{shares} \times \text{price}$, though the correlation isn't near 1.0 because it's a product of two independent variables, not a sum, and because some shares values are negative (see next point), which can suppress the linear correlation with amount.

Shares by txn_type (section 13) — the flag to prioritize

Sell and Dividend shares are all positive (1.00 to ~500), as expected. But Buy has 836 transactions (about 1.0% of all Buy rows) with negative share counts, down to -499.63. A “Buy” with negative shares doesn't have an obvious real-world meaning — it could be a sign-convention bug, a system logging a correction/cancellation as a Buy instead of a distinct reversal type, or a genuine data entry error. Before running any analysis that sums or aggregates shares by type, I'd pull those 836 rows out and look for a pattern (same advisor? same security? clustered on certain dates?) to figure out whether they should be dropped, treated as sells, or kept as-is. The NaN min/max for Advisory Fee/Deposit/Withdrawal here is expected, not an error — same root cause as the missing-values pattern in section 4, since those types have no shares at all.

Matplotlib warning

The warning at the bottom (MatplotlibDeprecationWarning: vert: bool was deprecated...) is just a library note about an upcoming matplotlib API change, unrelated to your data — the chart still renders correctly. Worth a quick code fix eventually (swap `vert=False` for `orientation="horizontal"`), but it's not a data issue.

Bottom line — one real data-quality flag to chase down: the 836 negative-share Buy transactions. Everything else (the “missing” security fields, the Advisory Fee mean/median gap, the low shares-price correlation) is either explainable by design or worth a note rather than a blocker.